

Chatbot Client – Travel Concierge Agent

A minimal JS client for the case loop. Four endpoints drive the whole conversation:

Action	Call
Start a case	POST /cases
Watch progress	GET /cases/{id} (poll)
Answer a question	POST /cases/{id}/messages
List past cases	GET /cases

The PNR must be embedded in the message text (e.g. "...booking PNR001...") — intake parses it from the sentence.

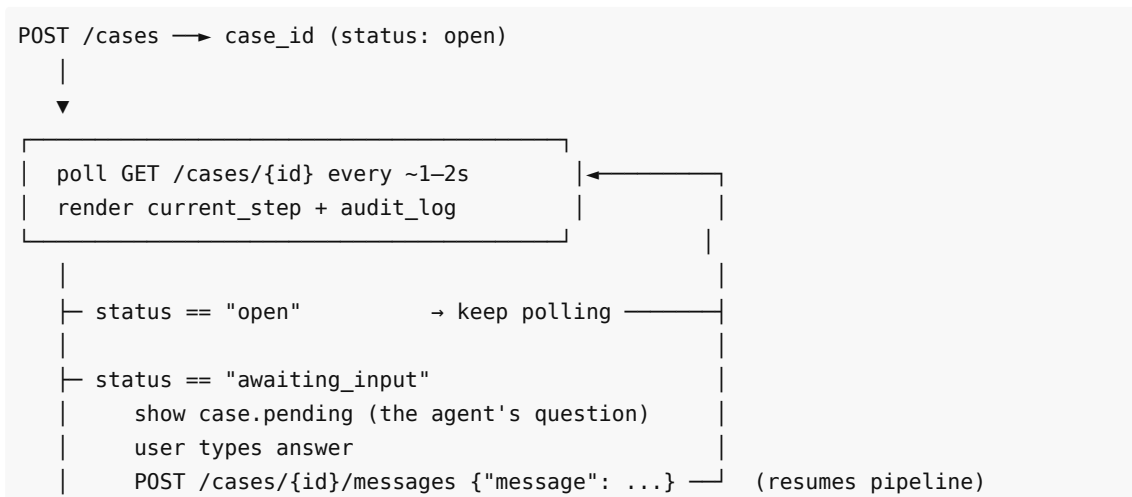
Integration overview

Only **4 endpoints** are needed for the conversational loop; the rest are reference/read-only data for the UI.

Core chatbot endpoints

Step	Endpoint	Body	Returns
Start	POST /cases	{"message": "...", "provider"?}	202 → {case_id, status, booking_ref, disruption_type}
Poll	GET /cases/{case_id}	—	full case: status, current_step, pending, slices, audit_log
Reply	POST /cases/{case_id}/messages	{"message": "..."} —	202 (only valid when status == awaiting_input)
History	GET /cases?limit=N	—	list of past cases

The loop (ASCII)



```
|
└─ status ∈ {closed, rebooked, compensated, failed}
    → terminal, stop polling, render final summary
```

Status values (`app/domain/enums.py:39`)

- `open` — pipeline running in background, keep polling
- `awaiting_input` — paused at an approval/question gate; `case.pending` holds the question to show the user → reply via `POST .../messages`
- `closed` / `rebooked` / `compensated` — terminal success states
- `failed` — terminal error

Why polling (not a single request)

`POST /cases` returns immediately with `202` and runs the multi-agent pipeline in a `BackgroundTask` (it uses a live LLM, so it's slow). The case row's `current_step`, `audit_log`, and `pending` are persisted as each agent step completes — so polling `GET /cases/{id}` is how the chatbot streams progress and detects the pause points.

Supporting endpoints for the UI (optional, read-only)

- `GET /demos` — prebuilt demo messages (with PNRs embedded)
- `GET /bookings` / `GET /bookings/{ref}` — booking picker
- `GET /providers` — cortex vs claude toggle
- `GET /health` — readiness check
- `GET /flights`, `/hotels`, `/airports`, `/rules`, `/claims`, `/customers` — world data for side panels

One nuance: the PNR must be embedded in the `message` text (e.g. `"...booking PNR001..."`) — intake parses it from the sentence, there's no separate field.

The loop

```
POST /cases → case_id (status: open)
|
▼ poll GET /cases/{id} every ~1.5s, render current_step + audit_log
|
├─ "open"          → keep polling
├─ "awaiting_input" → show case.pending, get user answer,
|                     POST /cases/{id}/messages, resume polling
└─ closed | rebooked | compensated | failed → terminal, stop
```

client.js

```
const BASE = "http://localhost:8000";

const TERMINAL = new Set(["closed", "rebooked", "compensated", "failed"]);

async function json(res) {
  if (!res.ok) throw new Error(`${res.status} ${await res.text()}`);
  return res.json();
}
```

```

}

const sleep = (ms) => new Promise((r) => setTimeout(r, ms));

/** Start a case. `message` must contain the PNR (e.g. "...booking PNR001..."). */
export function startCase(message, provider) {
  return fetch(`${BASE}/cases`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ message, ...(provider && { provider }) }),
  }).then(json); // -> { case_id, status, booking_ref, disruption_type }
}

export function getCase(caseId) {
  return fetch(`${BASE}/cases/${caseId}`).then(json);
}

/** Answer a pending question on a paused case. */
export function replyToCase(caseId, message) {
  return fetch(`${BASE}/cases/${caseId}/messages`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ message }),
  }).then(json);
}

/**
 * Drive a case to completion.
 * - onStep(case) : called on every poll where current_step changes
 * - askUser(question, case) -> Promise<string> : called when paused for input
 * Resolves with the final case object.
 */
export async function runCase(message, { provider, onStep, askUser } = {}) {
  const { case_id } = await startCase(message, provider);

  let lastStep = null;
  while (true) {
    const c = await getCase(case_id);

    if (c.current_step !== lastStep) {
      lastStep = c.current_step;
      onStep?.(c);
    }

    if (TERMINAL.has(c.status)) return c;

    if (c.status === "awaiting_input") {
      const answer = await askUser(c.pending, c); // block on the human
      await replyToCase(case_id, answer); // resume the pipeline
      // fall through and keep polling; status flips back to "open"
    }
  }
}

```

```

    await sleep(1500);
  }
}

```

Usage

```

import { runCase } from "./client.js";

const finalCase = await runCase(
  "My flight 6E2341 on booking PNR001 from Delhi to Mumbai tonight was cancelled.",
  {
    provider: "claude", // optional: "cortex" | "claude"
    onStep: (c) => console.log("step:", c.current_step, "-", c.status),
    askUser: async (question) => {
      // wire this to your chat input; here we just hardcode for a demo
      console.log("agent asks:", question);
      return prompt(question); // browser; or read from your UI
    },
  },
);

console.log("done:", finalCase.status);
console.log(finalCase.audit_log); // full step-by-step trace for the transcript

```

Minimal React hook

```

import { useState, useCallback, useRef } from "react";
import { startCase, getCase, replyToCase } from "./client.js";

const TERMINAL = new Set(["closed", "rebooked", "compensated", "failed"]);

export function useCase() {
  const [c, setCase] = useState(null);
  const idRef = useRef(null);

  const poll = useCallback(async () => {
    while (true) {
      const next = await getCase(idRef.current);
      setCase(next);
      if (TERMINAL.has(next.status) || next.status === "awaiting_input") return
    }
  }, []);

  const start = useCallback(async (message, provider) => {
    const { case_id } = await startCase(message, provider);
    idRef.current = case_id;
  }, []);
}

```

```

    return poll();
  }, [poll]);

  const reply = useCallback(async (message) => {
    await replyToCase(idRef.current, message); // status -> open
    return poll(); // resume watching
  }, [poll]);

  return { case: c, start, reply };
}

```

```

function Chat() {
  const { case: c, start, reply } = useCase();
  // call start("...PNR001...") on submit; when c.status === "awaiting_input"
  // render c.pending and call reply(answer) with the user's response.
  // render c.current_step / c.audit_log for live progress, c.slices for results.
}

```

Case shape (from GET /cases/{id})

Field	Meaning
case_id	identifier used in all follow-up calls
status	open awaiting_input rebooked compensated closed failed
current_step	skill currently running (null when finished)
pending	the agent's question — shown to the user when awaiting_input
slices	structured results produced by each agent (rebooking, compensation, ...)
audit_log	ordered trace of every step — render as the transcript
report	parsed intake (booking_ref, disruption_type, raw_text)

Notes

- POST /cases and POST .../messages return **202** immediately; work runs in a background task, so polling is required to follow progress.
- POST .../messages is only valid when status == "awaiting_input" — otherwise it returns **409**.
- Reference endpoints for UI side panels: GET /demos , /bookings , /providers , /flights , /hotels , /airports , /rules , /claims , /customers .
- Adjust BASE to match where the API server is bound.